

TianoForge: An Automated Bug Triage Approach for the TianoCore UEFI Firmware Development Community

Nazanin Siavash
nsiavash@uccs.edu
University of Colorado Colorado
Springs (UCCS)
Colorado, United States

Terrance E. Boulton
tboulton@uccs.edu
University of Colorado Colorado
Springs (UCCS)
Colorado, United States

Armin Moin
moin@purdue.edu
Purdue University
Indiana, United States

Abstract

We propose a novel approach to bug triage in the TianoCore open-source UEFI firmware development ecosystem. This integrated approach, called *TianoForge*, deploys the state of the art in artificial intelligence, specifically machine learning, to enable automated bug triage. This includes invalid bug report detection, duplicate bug report detection, bug report prioritization, and bug report assignment. We use various Generative Pretrained Transformer (GPT) Large Language Models (LLMs) with and without Retrieval Augmented Generation (RAG) to automate these tasks. Given the crucial role of bug triage in software maintenance and the huge number of untriaged issues in the TianoCore community, in particular, their primary project, EDK II, we expect a significant impact on the efficiency of TianoCore software maintenance processes, primarily bug triage and resolution. Our experimental study shows that TianoForge reduces the average bug triage time from around 11 days to approximately 7 minutes, which is a 99.95% reduction.

CCS Concepts

- **Software and its engineering** → **Software maintenance tools**;
- **Security and privacy** → *Software and application security*.

Keywords

bug triage, uefi, firmware, tianocore, edk ii, large language models

ACM Reference Format:

Nazanin Siavash, Terrance E. Boulton, and Armin Moin. 2026. TianoForge: An Automated Bug Triage Approach for the TianoCore UEFI Firmware Development Community. In *Proceedings of the 1st International Workshop on Firmware Testing and Analysis (FTA '26)*, October 04–09, 2026, Oakland, CA, USA. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3842651.3843187>

1 Introduction

Modern software systems evolve continuously to accommodate changing requirements and performance improvements, inevitably introducing defects [12, 19]. To manage these defects, bug tracking systems (e.g., Bugzilla) are widely used to collect and organize reports submitted by users and developers. These reports provide essential information for reproducing and resolving issues; however, efficiently processing them remains a significant challenge.

Bug triage, the process of evaluating reported issues and assigning them to suitable developers for resolution [3], is a critical yet resource-intensive task. In large-scale and open-source projects, the high volume of incoming reports makes manual triage impractical [4]. Delays in assignment, coupled with frequent reassignment (i.e., bug tossing), substantially increase resolution time and reduce overall efficiency. Moreover, accurate triage requires detailed knowledge of developer expertise and system components, which is difficult to maintain in dynamic, distributed teams [9]. As a result, manual triage often suffers from low accuracy and scalability limitations [27]. Automated bug triage has therefore become essential in modern software engineering [7, 24]. By leveraging data-driven and intelligent techniques, automated approaches can improve triage efficiency through report classification, prioritization, deduplication, and assignment [6, 16]. These methods reduce manual effort, shorten resolution cycles, and improve diagnostic precision by filtering noisy or redundant reports. Additionally, structured triage outputs enable downstream automation tasks, such as root cause analysis and failure prediction, supporting more proactive system maintenance. Beyond efficiency, the quality and trustworthiness of bug reports themselves present an emerging challenge. Recent discussions within the TianoCore EDK II community highlight concerns about undisclosed LLM-assisted bug reports, where AI-generated content introduced technical inaccuracies and undermined maintainer trust in issue-tracking data [25]. This underscores the importance of domain-aware automated triage tools that can assist developers in identifying and filtering low-quality or invalid reports, rather than relying solely on contributor-submitted content at face value.

Despite significant advances in automated bug triage, existing approaches primarily focus on one or two triage tasks in isolation, such as bug assignment, duplicate detection, or prioritization, rather than providing an integrated end-to-end triage solution. Moreover, most prior studies have been conducted on general-purpose software projects, while the TianoCore EDK II ecosystem has not previously been studied in the context of automated bug triage. As a result, the effectiveness of recent Large Language Model (LLM)-based approaches in integrated bug triage pipelines remains largely unexplored, particularly in the context of firmware development. Furthermore, the impact of retrieval augmentation, prompt engineering, and domain-specific knowledge on LLM-assisted triage performance is not yet well understood.

In this work, we propose *TianoForge*, an integrated automated bug triage script for the TianoCore EDK II ecosystem. TianoForge streamlines the bug triage process by automatically performing four key triage tasks: invalid detection, duplicate detection, priority



This work is licensed under a Creative Commons Attribution 4.0 International License. *FTA '26, Oakland, CA, USA*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2969-0/2026/10
<https://doi.org/10.1145/3842651.3843187>

prediction, and developer assignment within a unified workflow. We conduct an experimental study to address the following Research Questions (RQs): **RQ1**: Can automated bug triage reduce the bug resolution time compared to manual triage in TianoCore Projects, specifically EDK II? **RQ2**: How effective and efficient is the proposed automated triage framework across key sub-tasks (e.g., duplicate detection, prioritization, invalid report detection, and assignment)? **RQ3**: What are the key factors in deciding the priority level of the issues in TianoCore projects? **RQ4**: To what extent can prompt engineering improve the performance of LLM-based triage components?

The remainder of this paper is organized as follows. Section 2 provides background on bug triage, LLMs, and Retrieval-Augmented Generation (RAG). Section 3 reviews existing research on bug triage tasks. Section 4 presents TianoForge, including the problem formulation, architecture, and integrated triage workflow. Section 5 describes the experimental dataset, evaluation metrics, experimental setup, and results analysis. Section 6 discusses the main findings and implications of the results. Finally, Section 7 concludes the paper and outlines directions for future work.

2 Background

This section introduces the key concepts underlying the proposed automated bug triage framework. It first discusses bug triage and bug report enhancement, highlighting their role in managing software defects and improving issue quality. It then reviews LLMs and prompt engineering techniques that support automated reasoning over bug reports. Finally, it presents RAG, which enables LLMs to leverage external project-specific knowledge during inference.

2.1 Bug Triage and Report Enhancement

Bug tracking systems, such as Bugzilla, Jira, and GitHub Issues, are essential tools for managing software defects throughout the development lifecycle. These systems allow users and developers to report, track, and resolve issues in a structured manner. A typical bug report consists of a textual summary (title), a detailed description, and additional metadata such as priority, severity, component, and assigned developer. Over time, reports may also include comments, attachments, and status updates that reflect the debugging process.

The large volume of reports generated in modern software projects makes efficient issue management increasingly challenging. As a result, bug triage has become a critical process for ensuring that defects are analyzed and addressed effectively. Bug triage involves evaluating reported issues and determining appropriate actions, including invalid issue detection, duplicate detection, issue prioritization, and developer assignment [15]. These tasks help development teams allocate resources efficiently and reduce the time required to resolve reported issues.

The effectiveness of bug triage is highly dependent on the quality of the underlying bug reports. Incomplete descriptions, ambiguous titles, missing metadata, and redundant reports can hinder decision-making and reduce triage accuracy. To address these challenges, bug report enhancement techniques are often employed to improve report quality prior to or during the triage process. Such techniques may include refining issue titles, enriching reports with

additional context, and filtering invalid or duplicate submissions. By improving the quality of input reports, enhancement techniques can support more accurate and consistent triage decisions.

2.2 Large Language Models and Prompt Engineering

The growing complexity and scale of modern software repositories have motivated the development of automated approaches for bug triage. Recent advances in LLMs have made them promising candidates for supporting such automation. LLMs are transformer-based foundation models trained on massive text corpora to perform a wide range of natural language understanding and generation tasks [22]. Their ability to capture semantic relationships in unstructured text makes them well suited for analyzing bug reports and assisting with triage-related decisions.

In software engineering, LLMs have been applied to tasks such as code generation, software maintenance, defect prediction, and bug triage [18]. A key factor influencing their effectiveness is prompt engineering, which refers to the design of instructions and contextual information provided to the model. Techniques such as zero-shot and few-shot prompting allow LLMs to perform specialized tasks without additional model training. Furthermore, prompts can incorporate domain knowledge, decision criteria, and representative examples to guide model behavior and improve prediction quality [10].

2.3 Retrieval-Augmented Generation

While LLMs possess strong reasoning and language understanding capabilities, they are limited by the knowledge encoded in their training data. RAG addresses this limitation by incorporating external information during inference. Instead of relying solely on model parameters, RAG retrieves relevant documents from an external knowledge source and includes them as additional context in the prompt.

In bug triage applications, retrieval sources may include historical bug reports, project documentation, developer discussions, and previously resolved issues. By grounding predictions in project-specific information, RAG can provide additional context for tasks such as duplicate detection, prioritization, and developer assignment. Consequently, the combination of LLMs, prompt engineering, and retrieval augmentation has emerged as a promising direction for developing more effective automated bug triage systems.

3 Related Work

Bug triage is a fundamental task in software maintenance and closely interacts with related activities such as duplicate detection, prioritization, and bug validity assessment. Early research primarily formulated bug triage as a supervised text classification or optimization problem over bug report content and metadata [1]. Traditional approaches employed techniques such as Naïve Bayes classifiers, topic models, and bug tossing graphs to model developer expertise and report characteristics [5, 28, 29]. In recent years, research has shifted toward deep learning-based methods, representation learning, and hybrid architectures that leverage richer semantic and structural information from bug reports. Graph-based feature augmentation has been explored to enhance textual representations

of bug reports. For instance, Alazzam et al. constructed term graphs where words are treated as nodes and enriched via neighborhood relationships, significantly improving bug prioritization performance compared to traditional TF-IDF and baseline deep learning methods [2]. Similarly, Umer et al. [26] proposed a CNN-based prioritization framework that integrates syntactic, semantic, and emotional features, demonstrating substantial improvements in cross-project F1-scores and highlighting the importance of domain-specific signals in triage-related tasks. Hierarchical models have also been introduced to better capture the structure of bug reports. Yadav and Rathore [30] proposed a Hierarchical Attention Network (HAN) that models both word-level and sentence-level representations using DistilBERT tokenization and Bi-GRU layers. Their approach outperforms classical machine learning models, including SVM, random forest, and logistic regression, as well as standard deep learning baselines, demonstrating the effectiveness of hierarchical attention mechanisms. Beyond individual tasks, recent work has explored integrated pipelines for bug management. Chhabra and Chadha [8] introduced ReBaRF-Bug, which combines TF-IDF features, multi-stage data augmentation (e.g., SMOTE and paraphrasing), ResNet-based feature transformation, and ensemble learning. Their approach achieved high accuracy and robustness on datasets such as Eclipse and Mozilla, indicating that combining augmentation with deep feature extraction can effectively address data imbalance and sparsity. In specialized domains, such as security bug classification, cross-project learning, and similarity-based augmentation have been widely adopted. Ganzorig et al. [11] leveraged both lexical and embedding-based similarity methods to retrieve related bug reports from external projects and train deep learning models, including CNNs, LSTMs, GRUs, and Transformers. Their results demonstrated consistently high performance, although no single model dominated across all scenarios. More recently, Large Language Models (LLMs) have emerged as a promising paradigm for automated bug triage. Compared to traditional representations such as TF-IDF and static embeddings, transformer-based models provide richer contextual representations, leading to improved performance in tasks such as bug assignment and prioritization. These models are particularly effective when leveraging both bug titles and detailed descriptions. Dipongkor [14] investigated transformer-based LLMs for bug triaging and developer assignment, evaluating several models including BERT, RoBERTa, CodeBERT, and DeBERTa. Their study demonstrated that transformer-based models outperform traditional approaches and further showed that ensemble voting strategies can improve assignment accuracy beyond individual models. More recently, Kiashemshaki et al. [13] proposed an instruction-tuned LLM framework for automated bug triaging using a LoRA-adapted DeepSeek-R1 model with candidate-constrained decoding. Their approach generates ranked developer recommendations directly from bug reports without requiring handcrafted features or graph construction, demonstrating the feasibility of lightweight project-specific LLMs for practical bug assignment tasks. You et al. [31] proposed an “LLM-as-classifier” paradigm, introducing semi-supervised and human-in-the-loop strategies for hierarchical text classification, with applications including proactive bug triage and monitoring systems.

Hybrid architectures that combine LLMs with structural learning methods have also gained attention. Song et al. [23] proposed a

model that integrates LLM-derived representations with multi-scale feature fusion and graph neural networks (GNNs). By capturing both global semantic context and local structural relationships, their approach achieves consistent improvements across multiple evaluation metrics, suggesting strong potential for complex triage scenarios where relational information is important.

4 Proposed Approach

This section presents *TianoForge*, a unified LLM-based script for automated bug triage in TianoCore EDK II. TianoForge integrates invalid issue detection, duplicate detection, bug prioritization, and developer assignment within a single workflow.

4.1 Problem Formulation

Let $\mathcal{G} = \{b_1, b_2, \dots, b_N\}$ ($N = 75$) denote the set of GitHub-native bugs submitted to the EDK II repository, where each bug b_k is represented by the feature tuple

$$b_k = (\text{title}_k, \text{package}_k, \text{type}_k, \text{priority}_k, \text{comments}_k, \text{state}_k).$$

Let $\mathcal{G}^L \subseteq \mathcal{G}$ ($|\mathcal{G}^L| = 37$) denote the subset of bugs with a known first assignee. Let $\mathcal{B}$ denote the historical Bugzilla XML corpus used as a retrieval source in System B. Each task is solved by prompting an LLM in an in-context learning setting under two systems: **System A** and **System B**. The four tasks are described below in pipeline order.

Retrieval notation. All retrieval-augmented systems use a hybrid ranker combining a dense semantic retriever and a sparse lexical retriever. Let $\text{Dense}(b_k, \mathcal{X})$ denote the ranked list of bugs from corpus $\mathcal{X}$ ordered by cosine similarity to the BGE-base-en-v1.5 embedding of b_k , and let $\text{BM25}(b_k, \mathcal{X})$ denote the ranked list ordered by BM25 lexical score. Reciprocal Rank Fusion (RRF) combines both lists:

$$\text{RRF}(A, B)_i = \frac{1}{k_0 + \text{rank}_A(i)} + \frac{1}{k_0 + \text{rank}_B(i)}, \quad k_0 = 60,$$

where $\text{rank}_A(i)$ is the rank of item i in list A . The notation $\text{RRF}(\cdot, \cdot)_{1:k}$ denotes the top- k items by RRF score.

4.1.1 Invalid Bug Detection. Invalid bug detection aims to identify bug reports that do not correspond to genuine software defects.

Definition. Each bug $b_k \in \mathcal{G}$ is associated with a binary label $y_k^{\text{inv}} \in \{0, 1\}$. The objective is to predict:

$$f_{\text{inv}} : \mathcal{G} \rightarrow \{0, 1\}, \quad \hat{y}_k^{\text{inv}} = f_{\text{inv}}(b_k),$$

where $\hat{y}_k^{\text{inv}} = 1$ indicates the bug is *invalid* and $\hat{y}_k^{\text{inv}} = 0$ indicates the bug is *valid*.

Invalid Bug Criteria

A bug b_k is labeled invalid ($\hat{y}_k^{\text{inv}} = 1$) if it satisfies one or more of the following:

- (i) spam or entirely unrelated to firmware development;
 - (ii) a general usage question rather than a bug report or feature request;
 - (iii) describes behavior working as designed;
 - (iv) lacks sufficient information to reproduce or investigate.
- When in doubt, the LLM classifies the bug as *valid*.

LLM Predictor. The function f_{inv} receives $(\text{title}_k, \text{package}_k, \text{type}_k, \text{comments}_k, \text{state}_k)$ with the invalid state label masked to prevent leakage. **System A** provides $\kappa = 3$ fixed few-shot examples per class drawn from $\mathcal{B}$, identical across all queries. **System B** retrieves the top- $\kappa = 3$ most similar bugs from $\mathcal{B}$ per query via RRF:

$$\mathcal{C}_k = \text{RRF}(\text{Dense}(b_k, \mathcal{B}), \text{BM25}(b_k, \mathcal{B}))_{1:\kappa}, \quad \kappa = 3.$$

4.1.2 Duplicate Bug Detection. Duplicate bug detection determines whether a newly submitted bug describes an issue already reported in the repository.

Definition. Given a query bug $b_k \in \mathcal{G}$ and candidate bugs $\mathcal{C}_k \subset \mathcal{G} \setminus \{b_k\}$, an LLM f_{dup} predicts:

$$\hat{y}_k^{\text{dup}} = f_{\text{dup}}(b_k, \mathcal{C}_k) \in \{0, 1\},$$

where $\hat{y}_k^{\text{dup}} = 1$ indicates b_k is a duplicate of some $b_j \in \mathcal{C}_k$.

Candidate retrieval. Candidates are obtained via RRF over BGE-base-en-v1.5 and BM25, both indexed over $\mathcal{G}$:

$$\mathcal{C}_k = \text{RRF}(\text{Dense}(b_k, \mathcal{G}), \text{BM25}(b_k, \mathcal{G}))_{1:\kappa}, \quad \kappa = 5.$$

Duplicate Bug Criterion

Two bugs b_k and b_j are duplicates if and only if they describe the **same root cause** in the same codebase component. Topical similarity or a shared package is insufficient. When in doubt, the LLM returns $\hat{y}_k^{\text{dup}} = 0$.

LLM Predictor. Both systems retrieve candidate bugs $\mathcal{C}_k$ via RRF as described above. **System A** prompts the LLM with b_k and $\mathcal{C}_k$ only. **System B** additionally retrieves up to $\kappa' = 3$ confirmed duplicate pairs from $\mathcal{B}$ via RRF as in-context demonstrations.

4.1.3 Bug Prioritization. Bug priority classification automatically determines the urgency of a bug report, helping developers allocate resources efficiently.

Definition. Let $\mathcal{P} = \{\text{low}, \text{medium}, \text{high}\}$ be the set of priority levels. The task is formulated as a multi-class classification problem where an LLM f_{pri} predicts:

$$f_{\text{pri}} : \mathcal{G} \rightarrow \mathcal{P}, \quad \hat{y}_k^{\text{pri}} = \arg \max_{p_j \in \mathcal{P}} P(p_j | b_k)$$

Priority Level Definitions

high

Crashes, data corruption, security vulnerabilities within the UEFI threat model, or build failures blocking *all* configurations for all users.

medium

Functional bugs with workarounds, compiler-specific build failures, or missing features required for spec compliance.

low

Cosmetic defects, code-quality improvements, warnings not blocking builds, optional features, or documentation changes.

LLM Predictor The function f_{pri} is prompted with b_k 's features and domain-calibration rules encoding EDK II-specific priority heuristics (e.g., CVE patches are not automatically high under

the UEFI threat model; compiler-specific build failures are medium). **System A** uses Leave-One-Out (LOO): all $N - 1$ bugs in $\mathcal{G}$ with known priority labels serve as in-context examples. **System B** uses the same LOO examples as System A and additionally retrieves the top- κ Bugzilla bugs with known priorities via RRF:

$$\mathcal{R}_k = \text{RRF}(\text{Dense}(b_k, \mathcal{B}^{\mathcal{P}}), \text{BM25}(b_k, \mathcal{B}^{\mathcal{P}}))_{1:\kappa}, \quad \kappa = 5,$$

where $\mathcal{B}^{\mathcal{P}} \subseteq \mathcal{B}$ is the Bugzilla corpus filtered to bugs with *priority* $\in \mathcal{P}$.

4.1.4 Bug Assignment. Bug assignment aims to automatically recommend/predict the most appropriate developer for resolving a newly submitted bug.

Definition. Let $\mathcal{D} = \{d_1, d_2, \dots, d_{|\mathcal{D}|}\}$ be the closed set of developers observed in $\mathcal{G}^L$. The task is formulated as a multi-class classification problem where an LLM f_{asgn} predicts:

$$f_{\text{asgn}} : \mathcal{G}^L \rightarrow \mathcal{D}, \quad \hat{y}_k^{\text{asgn}} = \arg \max_{d_j \in \mathcal{D}} P(d_j | b_k).$$

Assignment Constraint

The predicted assignee must belong to the closed set $\mathcal{D}$. Predictions outside $\mathcal{D}$ are corrected by case-insensitive exact matching; if no match is found, the prediction defaults to the first assignee in $\mathcal{D}$. The LLM is provided with package-level assignment heuristics encoding domain knowledge of the edk2 codebase (e.g., CryptoPkg CLANG build errors $\rightarrow$ mdkinney; NetworkPkg CodeQL bugs $\rightarrow$ BritChesley; OvmfPkg physical memory bugs $\rightarrow$ os-d).

LLM Predictor The function f_{asgn} is prompted with b_k 's features, package-level heuristics, and in-context labelled examples. **System A** uses LOO: the prompt contains all $|\mathcal{G}^L| - 1$ labelled examples from $\mathcal{G}^L$ excluding b_k . **System B** retrieves the top- κ most similar bugs from $\mathcal{G}^L$ via RRF:

$$\mathcal{S}_k = \text{RRF}(\text{Dense}(b_k, \mathcal{G}^L), \text{BM25}(b_k, \mathcal{G}^L))_{1:\kappa}, \quad \kappa = 5.$$

4.1.5 Integrated Triage Script. The four tasks above are composed into a single sequential script operating on $\mathcal{G}^L$, enabling end-to-end triage from submission to developer assignment.

Design The framework shares a single ChromaDB vector store and BM25 index across all four tasks, eliminating redundant indexing. A unified `call_llm` function routes requests to the OpenAI Chat Completions API, OpenAI Responses API (GPT-5+), or Anthropic Messages API based on the model name. The best-performing model for each task is active by default: **System A** with `claude-sonnet-4-6` for Tasks 1–3 and `gpt-5.5` for Task 4.

Output. For each $b_k \in \mathcal{G}^L$: $\hat{y}_k^{\text{inv}} \in \{0, 1\}$, $\hat{y}_k^{\text{dup}} \in \{0, 1\}$, $\hat{y}_k^{\text{pri}} \in \mathcal{P}$, $\hat{y}_k^{\text{asgn}} \in \mathcal{D}$.

4.2 Architecture of the Proposed Solution

Figure 1 presents the architecture of **TianoForge**, an integrated automated bug triage script for the TianoCore EDK II ecosystem. TianoForge combines four key triage tasks, invalid detection, duplicate detection, priority prediction, and developer assignment, into a unified sequential workflow that automatically processes submitted bug reports and produces a complete triage record. The architecture consists of three main components: (1) data sources,

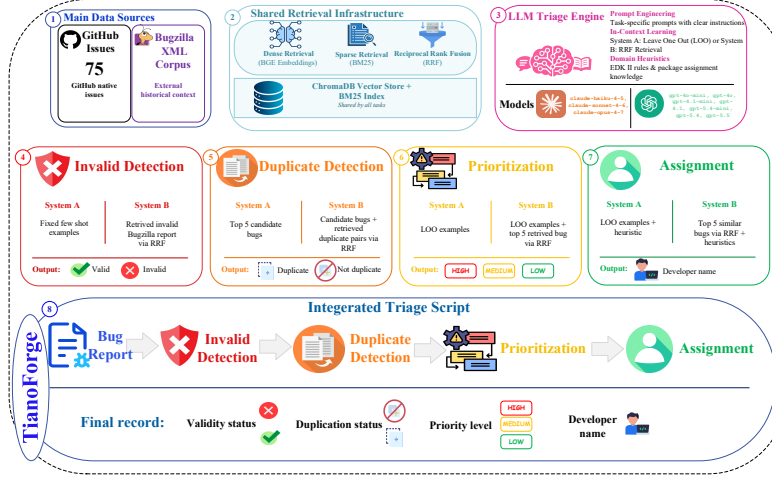


Figure 1: Overview of TianoForge

(2) a shared retrieval and reasoning layer, and (3) task-specific triage modules. The primary input is a GitHub-native bug report, while a historical Bugzilla XML corpus serves as an external knowledge source. The GitHub dataset contains the issues being triaged, whereas the Bugzilla corpus provides historical context used for retrieval-augmented prompting RAG configurations.

To support retrieval-augmented triage, TianoForge employs a shared retrieval infrastructure consisting of a dense semantic retriever based on BGE-base-en-v1.5 embeddings, a sparse lexical retriever based on BM25, and Reciprocal Rank Fusion (RRF) for combining retrieval results. Both retrieval mechanisms operate over a shared ChromaDB vector store and BM25 index constructed from the historical Bugzilla corpus. This shared infrastructure eliminates redundant indexing and enables all triage tasks to access relevant historical information through a common retrieval layer.

The retrieved context is consumed by a unified LLM triage engine responsible for executing all four triage tasks. The engine incorporates task-specific prompt engineering, in-context learning, and domain heuristics derived from the EDK II ecosystem. Depending on the task and experimental configuration, prompts are executed using models from the OpenAI and Anthropic families.

The first stage of the script performs *invalid detection*. Given a bug report, the model determines whether the issue represents a valid software defect or should be considered invalid. System A relies on fixed few-shot examples, whereas System B augments the prompt with retrieved invalid reports from the Bugzilla corpus.

The second stage performs *duplicate detection*. Candidate duplicate reports are first retrieved using the hybrid retrieval layer. System A uses only the retrieved candidate bugs, while System B additionally incorporates retrieved duplicate pairs from the Bugzilla XML corpus as contextual examples. The output is a binary decision indicating whether the issue is a duplicate of an existing report.

The third stage performs *prioritization*. This component classifies issues into low, medium, or high priority levels using domain-calibrated EDK II priority criteria. System A utilizes LOO examples,

while System B supplements these examples with top-ranked historical bugs retrieved from the Bugzilla corpus. The resulting priority label reflects the urgency. The final stage performs *developer assignment*. Using package-level assignment heuristics and historical examples, the model predicts the most suitable developer to resolve the issue. System A relies on LOO examples and assignment heuristics, whereas System B retrieves the top- κ most similar bugs from the labeled GitHub-native subset $\mathcal{G}^L$ via RRF as in-context demonstrations.

The four tasks are executed sequentially within the **TianoForge** script. Starting from a bug report, the script produces a final triage record consisting of the validity status, duplicate status, priority level, and recommended developer. By integrating these traditionally independent triage activities into a single automated workflow, TianoForge enables rapid and consistent bug triage while leveraging both project-specific domain knowledge and historical repository information.

5 Experimental Study

5.1 Experimental Dataset

The EDK II dataset is constructed from the publicly available issue tracking system of the TianoCore EDK II project hosted on GitHub. To the best of our knowledge, this dataset has not been previously utilized in the context of this research problem. The dataset is collected and curated on **April 3, 2026**, ensuring that the repository state is fixed at that point in time.

Data collection is performed programmatically using the GitHub REST API through a custom Python script, retrieving issues in a paginated manner with the following constraints: (i) only issues (excluding pull requests) are retrieved, (ii) only closed issues are included, and (iii) only issues labelled as `type: bug` are selected.

The final dataset consists of 2,610 issues structured as a CSV file with the following fields: issue number, title, state, first assignee, first assignment timestamp, triage hours, creation and closure timestamps, URL, milestone, comment count, and three label-derived categorical features: package, priority, and state. The dataset

comprises two subsets: 2,535 *Bugzilla-transferred issues* (historical bugs migrated from the legacy Bugzilla tracker) and 75 *GitHub-native issues* (bugs filed directly on GitHub). The GitHub-native issues span issues created between December 2024 and February 2026. Their priority labels are distributed as 32 medium, 28 low, and 15 high. Of the 75 GitHub-native issues, 37 carry a known first assignee and form the labelled subset $\mathcal{G}^L$ used for supervised evaluation of bug assignment.

5.2 Evaluation Metrics

All four tasks are evaluated using standard classification metrics. Let y_k denote the ground truth label and $\hat{y}_k$ the predicted label for bug b_k . For a given class c , let TP_c , FP_c , FN_c , and TN_c denote the number of True Positives, False Positives, False Negatives, and True Negatives, respectively.

5.2.1 Accuracy. measures the fraction of correctly classified instances over all n evaluated bugs:

$$\text{Acc} = \frac{1}{n} \sum_{k=1}^n 1[\hat{y}_k = y_k]$$

5.2.2 Precision. for class c measures the fraction of predicted positives that are truly positive:

$$P_c = \frac{TP_c}{TP_c + FP_c}.$$

5.2.3 Recall. for class c measures the fraction of true positives that are correctly identified:

$$R_c = \frac{TP_c}{TP_c + FN_c}.$$

5.2.4 F1-Score. for class c is the harmonic mean of precision and recall:

$$F1_c = \frac{2 \cdot P_c \cdot R_c}{P_c + R_c}$$

For all four tasks, two aggregation strategies are reported.

Macro averaging computes the unweighted mean across all classes:

$$P_{\text{macro}} = \frac{1}{|C|} \sum_{c \in C} P_c, \quad R_{\text{macro}} = \frac{1}{|C|} \sum_{c \in C} R_c, \quad F1_{\text{macro}} = \frac{1}{|C|} \sum_{c \in C} F1_c$$

Weighted averaging weights each class by its support n_c (number of true instances of class c):

$$P_{\text{weighted}} = \frac{1}{n} \sum_{c \in C} n_c P_c, \quad R_{\text{weighted}} = \frac{1}{n} \sum_{c \in C} n_c R_c, \\ F1_{\text{weighted}} = \frac{1}{n} \sum_{c \in C} n_c F1_c.$$

5.2.5 Resolution Time. provides a practical efficiency metric complementing the classification measures above. The total resolution time of a bug b_k is defined as:

$$T_k^{\text{res}} = T_k^{\text{triage}} + T_k^{\text{fix}},$$

where T_k^{triage} is the *triage time* (elapsed time from issue creation to first assignment) and T_k^{fix} is the *fix time* (elapsed time from first assignment to issue closure). Since T_k^{fix} depends on developer effort and is unaffected by the triage pipeline, reducing T_k^{triage} directly reduces T_k^{res} .

The mean triage time across the 37 labeled GitHub-native issues $\mathcal{G}^L$ is $\bar{T}^{\text{triage}} = 260.57$ hours (10.86 days), reflecting the manual triage latency in the current process. We compare this against the elapsed time of the integrated triage script measured using Python's `time.time()`, which records three intervals: setup time (CSV loading and index construction), inference time (LLM API calls across all four tasks), and total elapsed time (setup + inference). Each measurement is averaged over three independent runs to account for the non-deterministic nature of LLM inference. TianoForge produces all four predictions for all 37 issues in a single automated run, and any reduction in triage time directly translates to a reduction in overall resolution time T_k^{res} .

5.3 Experimental Setup

All experiments are conducted on Google Colab. The pipeline is implemented in Python and relies on the following key libraries: `openai`, `anthropic`, `chromadb`, `sentence-transformers`, `rank_bm25`, `scikit-learn`, and `tqdm`.

Ten LLMs are evaluated across all tasks, spanning two providers, OpenAI and Anthropic. For inference, temperature is set to 0.0 for all OpenAI models to ensure deterministic outputs. Claude models are called with default settings as the temperature parameter. For retrieval, we use BAAI/bge-base-en-v1.5 as the dense embedding model, indexed via ChromaDB with cosine similarity. Sparse retrieval uses BM25Okapi. Both are combined via Reciprocal Rank Fusion with $k_0 = 60$. The top- κ values are $\kappa = 5$ for duplicate candidates, priority context, and assignment context, and $\kappa' = 3$ for invalid detection context and duplicate pair demonstrations.

5.4 Experimental Analysis

This section presents the experimental results for each triage sub-task and the integrated script. All results are averaged over three independent runs to account for LLM non-determinism.

5.4.1 Duplicate Bug Detection. Table 1 reports results under the zero-duplicate ground truth, where no labeled duplicate pairs exist among the 75 GitHub-native bugs. Under this assumption, a model that never flags any issue achieves perfect positive-class precision ($P_+ = 1.0$), and accuracy reflects the fraction of issues correctly left unflagged. Three models namely gpt-4o-mini System A, claude-sonnet-4-6 System A/B, and claude-opus-4-7 System A flag zero issues consistently across all three runs, achieving $\text{Acc} = 0.973$ and $\text{Mac-F1} = 0.493$. claude-sonnet-4-6 is the most robust, maintaining this conservative behaviour on both System A and System B, while most other models produce false positives on System B.

Table 2 reports results under the pair-based ground truth comprising two confirmed duplicate pairs: {11791, 11790} and {11948, 11462}. These pairs are identified through a manual review process that goes beyond the scope of automated bug triage: we inspect every issue flagged as a duplicate by the LLMs across all runs and models, and manually verify whether the flag is correct by reading the issue content and comparing it against the reported original. Issues confirmed as genuine duplicates are added to the ground truth. This process serves a dual purpose; it enables a more meaningful evaluation of duplicate detection performance, and

it contributes to the *data quality improvement* of the TianoCore EDK II repository.

Evaluation is pair-based, meaning a true positive is scored when either member of a pair is correctly flagged as a duplicate of the other. All models achieve $R+ = 0.500$, meaning every model detects exactly one of the two pairs on average. The differentiating factor is false positives. On System A, three models achieve $FP = 0$ across all runs — gpt-4o-mini, claude-sonnet-4-6, and claude-opus-4-7 — yielding $P+ = 1.000$, $Mac-F1 = 0.826$, and $Acc = 0.960$. On System B, claude-sonnet-4-6 is the only model achieving $FP = 0$. System A consistently outperforms System B for most models, as Bugzilla RAG context generally increases false positives rather than reducing them, suggesting that historical duplicate pair examples from Bugzilla do not transfer well to the GitHub issue domain.

5.4.2 Invalid Bug Detection. Under this extended ground truth, only claude-sonnet-4-6 System A successfully identifies any of the four confirmed invalid bugs, catching one per run on average ($R+ = 0.250$, $F1+ = 0.333$). This translates to the best $Mac-F1$ (0.653) and the best performance across all metrics, including $Acc = 0.947$ and $Wgt-F1 = 0.938$. System A outperforms System B for Claude models, while System B improves GPT models’ false positive behaviour, suggesting the benefit of RAG is model-dependent for this task.

5.4.3 Bug Prioritization. Table 3 reports priority classification results across all ten models and both systems. No single model wins across all metrics. gpt-5.4-mini System A achieves the highest accuracy ($Acc = 0.671$) and weighted metrics ($Wgt-F1 = 0.663$), reflecting better performance on the majority class (medium). claude-sonnet-4-6 System A achieves the highest macro-recall ($Mac-R = 0.616$) and macro-F1 ($Mac-F1 = 0.620$), reflecting more balanced predictions across all three priority levels.

Across all models, System A consistently outperforms System B by a substantial margin. For example, gpt-5.4-mini achieves $Acc = 0.671$ under System A versus 0.542 under System B, and claude-sonnet-4-6 achieves $Acc = 0.636$ versus 0.556. This pattern holds across all ten models, confirming that the LOO in-context learning strategy of System A, which uses GitHub examples exclusively, is more effective than Bugzilla-augmented prompting for this task.

5.4.4 Bug Assignment. Table 4 reports bug assignment results on the 37 labeled issues with known first assignees. gpt-5.5 System A achieves the best performance across every metric: $Acc = 0.973$, $Mac-F1 = 0.977$, and $Wgt-F1 = 0.973$. This represents a substantial improvement over all other models, with the second-best being gpt-5.4 System A ($Acc = 0.883$, $Mac-F1 = 0.851$). Claude models, while competitive, lag behind the top GPT models on this task. Table 5 summarizes all the best-performing model and configuration for each triage task.

5.4.5 Integrated Triage Script. Table 6 presents results of the integrated script, applying all four tasks sequentially to the same 37 labeled issues using the best-performing model for each task under System A: claude-sonnet-4-6 for Tasks 1–3 and gpt-5.5 for Task 4 (the selected best-performing model for each task is active by default in TianoForge, while all other models and configurations remain available in the integrated codebase.) Since no confirmed invalid bugs or duplicate pairs exist within the 37 labeled issues, Tasks 1 and 2 operate under the zero ground-truth assumption.

Duplicate detection achieves perfect scores ($Acc = 1.000$, $Mac-F1 = 1.000$), confirming that claude-sonnet-4-6 correctly avoids false duplicate flags on this set. Invalid detection achieves $Acc = 0.973$ with one false positive on average. Priority classification in the integrated setting achieves $Acc = 0.559$ and $Mac-F1 = 0.487$, lower than the standalone results on 75 issues ($Acc = 0.636$, $Mac-F1 = 0.620$). This reduction is expected since the 37 labeled issues represent a smaller and different distribution than the full 75-issue set. Bug assignment achieves $Acc = 0.955$ and $Mac-F1 = 0.946$, slightly below the standalone result ($Acc = 0.973$).

Table 7 reports the runtime of the integrated pipeline averaged over three runs. The total pipeline completes in 424.61 seconds (7.08 minutes) on average, with LLM inference accounting for 419.20 seconds and setup taking only 5.40 seconds since the ChromaDB vector index is pre-built and reused across runs. Bug assignment is the most time-consuming task (145.39 s) due to the large LOO prompt containing 36 labeled examples per query.

The historical average triage time for the same 37 labeled issues, derived from the Triage Hours field in the dataset, amounts to 10.86 days. The integrated script reduces this to 7.08 minutes on average over 3 runs, representing a reduction of **99.95%**, more than three orders of magnitude ($\approx 2,208\times$ speedup). Since triage is the first and often most time-consuming step before a bug can be assigned and acted upon, reducing triage time directly leads to a reduction in overall bug resolution time. This demonstrates the potential of LLM-based automated triage to dramatically accelerate the bug resolution process in the TianoCore EDK II project.

6 Discussion

RQ1: Can automated bug triage reduce the bug resolution time compared to manual triage? The results indicate that the proposed solution substantially reduces triage latency. The integrated triage script processes all four triage tasks for the 37 labeled GitHub-native issues in an average of 7.08 minutes, compared to an average manual triage time of 260.57 hours (10.86 days). This corresponds to a 99.95% reduction in triage time, or approximately a 2,208 $\times$ speedup. Since total bug resolution time is defined as $T_k^{res} = T_k^{triage} + T_k^{fix}$, and the framework only affects the triage component, this reduction directly contributes to faster issue resolution.

RQ2: How effective and efficient is the proposed automated triage framework across key sub-tasks? The effectiveness of the proposed framework varies across triage tasks. *Bug assignment* achieves the strongest performance, with gpt-5.5 System A obtaining an accuracy of 0.973 and a macro-F1 score of 0.977. This result suggests that combining leave-one-out in-context examples with package-specific assignment heuristics effectively captures developer expertise within the EDK II ecosystem. *Duplicate detection* also performs well, with claude-sonnet-4-6 achieving perfect precision ($P+ = 1.000$) under the pair-based ground truth while producing no false positives. *Bug prioritization* achieves moderate performance (best macro-F1 = 0.620), likely reflecting the subjective nature of priority assignment. *Invalid issue detection* remains the most challenging task. Under the extended ground truth, only claude-sonnet-4-6 System A successfully identifies any invalid reports ($R+ = 0.250$, $macro-F1 = 0.653$). Beyond classification performance, the framework also contributes to repository quality

Table 1: Duplicate Detection Results — Average over 3 Runs (Ground Truth: Zero Duplicates)

Model	Sys	P+	R+	F1+	P-	R-	F1-	Mac-P	Mac-R	Mac-F1	Wgt-P	Wgt-R	Wgt-F1	Acc
gpt-4o-mini	A	1.000	1.000	1.000	1.000	0.973	0.987	1.000	0.987	0.493	1.000	0.973	0.987	0.973
	B	0.000	1.000	0.000	1.000	0.924	0.961	0.500	0.962	0.480	1.000	0.924	0.961	0.924
gpt-4o	A	0.000	1.000	0.000	1.000	0.938	0.968	0.500	0.969	0.484	1.000	0.938	0.968	0.938
	B	0.000	1.000	0.000	1.000	0.742	0.852	0.500	0.871	0.426	1.000	0.742	0.852	0.742
gpt-4.1-mini	A	0.000	1.000	0.000	1.000	0.947	0.973	0.500	0.973	0.486	1.000	0.947	0.973	0.947
	B	0.000	1.000	0.000	1.000	0.907	0.951	0.500	0.953	0.476	1.000	0.907	0.951	0.907
gpt-4.1	A	0.000	1.000	0.000	1.000	0.960	0.980	0.500	0.980	0.490	1.000	0.960	0.980	0.960
	B	0.000	1.000	0.000	1.000	0.893	0.944	0.500	0.947	0.472	1.000	0.893	0.944	0.893
gpt-5.4-mini	A	0.000	1.000	0.000	1.000	0.884	0.939	0.500	0.942	0.469	1.000	0.884	0.939	0.884
	B	0.000	1.000	0.000	1.000	0.942	0.970	0.500	0.971	0.485	1.000	0.942	0.970	0.942
gpt-5.4	A	0.000	1.000	0.000	1.000	0.920	0.958	0.500	0.960	0.479	1.000	0.920	0.958	0.920
	B	0.000	1.000	0.000	1.000	0.902	0.949	0.500	0.951	0.474	1.000	0.902	0.949	0.902
gpt-5.5	A	0.000	1.000	0.000	1.000	0.956	0.977	0.500	0.978	0.489	1.000	0.956	0.977	0.956
	B	0.000	1.000	0.000	1.000	0.924	0.961	0.500	0.962	0.480	1.000	0.924	0.961	0.924
claude-haiku-4-5	A	0.000	1.000	0.000	1.000	0.885	0.939	0.500	0.942	0.469	1.000	0.885	0.939	0.885
	B	0.000	1.000	0.000	1.000	0.951	0.975	0.500	0.976	0.488	1.000	0.951	0.975	0.951
claude-sonnet-4-6	A	1.000	1.000	1.000	1.000	0.973	0.987	1.000	0.987	0.493	1.000	0.973	0.987	0.973
	B	1.000	1.000	1.000	1.000	0.973	0.987	1.000	0.987	0.493	1.000	0.973	0.987	0.973
claude-opus-4-7	A	1.000	1.000	1.000	1.000	0.973	0.987	1.000	0.987	0.493	1.000	0.973	0.987	0.973
	B	0.000	1.000	0.000	1.000	0.960	0.980	0.500	0.980	0.490	1.000	0.960	0.980	0.960

Table 2: Duplicate Detection Results — Average over 3 Runs (Ground Truth: 2 Pairs, {11791,11790} and {11948,11462})

Model	Sys	P+	R+	F1+	P-	R-	F1-	Mac-P	Mac-R	Mac-F1	Wgt-P	Wgt-R	Wgt-F1	Acc
gpt-4o-mini	A	1.000	0.500	0.667	0.973	1.000	0.986	0.986	0.750	0.826	0.973	0.986	0.977	0.960
	B	0.217	0.500	0.302	0.971	0.948	0.960	0.594	0.724	0.631	0.951	0.936	0.942	0.911
gpt-4o	A	0.278	0.500	0.356	0.972	0.962	0.967	0.625	0.731	0.661	0.953	0.950	0.950	0.924
	B	0.072	0.667	0.129	0.976	0.762	0.856	0.524	0.714	0.492	0.952	0.759	0.836	0.742
gpt-4.1-mini	A	0.333	0.500	0.400	0.972	0.972	0.972	0.653	0.736	0.686	0.954	0.959	0.956	0.933
	B	0.170	0.500	0.253	0.971	0.930	0.950	0.570	0.715	0.601	0.949	0.918	0.931	0.893
gpt-4.1	A	0.500	0.500	0.500	0.972	0.986	0.979	0.736	0.743	0.740	0.959	0.973	0.966	0.947
	B	0.143	0.500	0.222	0.970	0.916	0.942	0.557	0.708	0.582	0.948	0.904	0.922	0.880
gpt-5.4-mini	A	0.137	0.500	0.211	0.970	0.902	0.934	0.553	0.701	0.573	0.947	0.891	0.915	0.871
	B	0.306	0.500	0.378	0.972	0.967	0.969	0.639	0.734	0.674	0.953	0.954	0.953	0.929
gpt-5.4	A	0.200	0.500	0.286	0.971	0.944	0.957	0.586	0.722	0.621	0.950	0.932	0.939	0.907
	B	0.159	0.500	0.241	0.970	0.925	0.947	0.565	0.712	0.594	0.948	0.913	0.928	0.889
gpt-5.5	A	0.444	0.500	0.467	0.972	0.981	0.977	0.708	0.741	0.722	0.958	0.968	0.963	0.942
	B	0.217	0.500	0.302	0.971	0.948	0.960	0.594	0.724	0.631	0.951	0.936	0.942	0.911
claude-haiku-4-5	A	0.141	0.500	0.216	0.970	0.906	0.937	0.555	0.703	0.577	0.947	0.895	0.917	0.871
	B	0.389	0.500	0.433	0.972	0.977	0.974	0.680	0.738	0.704	0.956	0.964	0.959	0.938
claude-sonnet-4-6	A	1.000	0.500	0.667	0.973	1.000	0.986	0.986	0.750	0.826	0.973	0.986	0.977	0.960
	B	1.000	0.500	0.667	0.973	1.000	0.986	0.986	0.750	0.826	0.973	0.986	0.977	0.960
claude-opus-4-7	A	1.000	0.500	0.667	0.973	1.000	0.986	0.986	0.750	0.826	0.973	0.986	0.977	0.960
	B	0.500	0.500	0.500	0.972	0.986	0.979	0.736	0.743	0.740	0.959	0.973	0.966	0.947

Table 3: Bug Prioritization Results — Average over 3 Runs (75 Issues)

Model	Sys	Acc	Mac-P	Mac-R	Mac-F1	Wgt-P	Wgt-R	Wgt-F1
gpt-4o-mini	A	0.622	0.638	0.582	0.591	0.649	0.622	0.614
	B	0.498	0.463	0.446	0.447	0.490	0.498	0.487
gpt-4o	A	0.587	0.561	0.529	0.534	0.595	0.587	0.578
	B	0.556	0.571	0.508	0.515	0.563	0.556	0.544
gpt-4.1-mini	A	0.578	0.555	0.533	0.537	0.587	0.578	0.574
	B	0.467	0.444	0.424	0.423	0.461	0.467	0.456
gpt-4.1	A	0.627	0.602	0.598	0.597	0.638	0.627	0.629
	B	0.551	0.515	0.505	0.498	0.553	0.551	0.540
gpt-5.4-mini	A	0.671	0.628	0.614	0.617	0.662	0.671	0.663
	B	0.542	0.483	0.483	0.468	0.521	0.542	0.518
gpt-5.4	A	0.649	0.614	0.590	0.592	0.635	0.649	0.635
	B	0.507	0.489	0.473	0.465	0.520	0.507	0.495
gpt-5.5	A	0.622	0.589	0.553	0.553	0.613	0.622	0.603
	B	0.498	0.471	0.453	0.435	0.500	0.498	0.473
claude-haiku-4-5	A	0.591	0.608	0.577	0.578	0.623	0.591	0.590
	B	0.560	0.541	0.542	0.541	0.562	0.560	0.560
claude-sonnet-4-6	A	0.636	0.636	0.616	0.620	0.652	0.636	0.637
	B	0.556	0.523	0.511	0.510	0.543	0.556	0.544
claude-opus-4-7	A	0.533	0.547	0.507	0.513	0.564	0.533	0.533
	B	0.551	0.553	0.526	0.530	0.555	0.551	0.545

improvement. Manual verification of LLM-flagged reports uncovers three previously unlabeled invalid issues and two duplicate pairs, demonstrating that the framework can assist not only in automated triage but also in improving the quality of issue-tracking data.

RQ3: What are the key factors in deciding the priority level of issues in TianoCore projects? The priority criteria necessary for accurate LLM-based prioritization are identified through iterative prompt engineering: the heuristics that must be explicitly encoded to achieve correct predictions reveal what drives priority assignment in EDK II in practice. Three factors prove necessary: impact scope (crashes, data corruption, and all-configuration build failures map to high; compiler-specific or workaround-available failures to medium), security within the UEFI threat model (CVE-adjacent reports are not automatically high priority; exploitability within the firmware threat model is the relevant criterion), and defect type (cosmetic issues, documentation changes, and optional features map to low, as models otherwise default to medium in ambiguous cases). However, the moderate classification results (best Mac-F1 =

Table 4: Bug Assignment Results — Average over 3 Runs (37 labeled issues)

Model	Sys	Acc	P-mac	R-mac	F1-mac	P-wgt	R-wgt	F1-wgt
gpt-4o-mini	A	0.784	0.710	0.764	0.724	0.745	0.784	0.751
	B	0.802	0.757	0.795	0.765	0.768	0.802	0.771
gpt-4o	A	0.838	0.773	0.810	0.781	0.809	0.838	0.811
	B	0.838	0.767	0.810	0.782	0.791	0.838	0.808
gpt-4.1-mini	A	0.811	0.753	0.793	0.755	0.779	0.811	0.774
	B	0.811	0.751	0.793	0.753	0.778	0.811	0.773
gpt-4.1	A	0.838	0.767	0.810	0.777	0.804	0.838	0.808
	B	0.838	0.761	0.810	0.777	0.786	0.838	0.804
gpt-5.4-mini	A	0.838	0.770	0.810	0.780	0.784	0.838	0.798
	B	0.838	0.765	0.810	0.776	0.788	0.838	0.800
gpt-5.4	A	0.883	0.851	0.868	0.851	0.883	0.883	0.874
	B	0.838	0.790	0.810	0.791	0.831	0.838	0.825
gpt-5.5	A	0.973	0.983	0.983	0.977	0.986	0.973	0.973
	B	0.883	0.828	0.868	0.834	0.866	0.883	0.861
claude-haiku-4-5	A	0.847	0.781	0.822	0.791	0.793	0.847	0.807
	B	0.838	0.753	0.810	0.770	0.766	0.838	0.789
claude-sonnet-4-6	A	0.856	0.771	0.822	0.789	0.797	0.856	0.819
	B	0.838	0.750	0.810	0.771	0.777	0.838	0.799
claude-opus-4-7	A	0.838	0.764	0.810	0.775	0.797	0.838	0.804
	B	0.838	0.762	0.810	0.774	0.791	0.838	0.801

0.620) suggest that these factors are necessary but not sufficient, as priority assignment in EDK II likely involves additional implicit criteria such as reporter identity, milestone urgency, or cross-package dependencies that are not captured in the available issue metadata. The degradation observed under System B, which augments the same domain-calibrated prompt with retrieved Bugzilla EDK II examples, further indicates that the priority conventions used in the legacy Bugzilla tracker do not align well with those of the GitHub-native issues, introducing noise rather than useful signal.

RQ4: To what extent can prompt engineering improve the performance of LLM-based triage components? The results demonstrate that prompt engineering has a measurable effect on triage performance, though retrieval augmentation does not consistently improve results. For prioritization and assignment, System A outperforms System B across all ten models, with accuracy gaps of up to 0.129 points for prioritization (e.g., gpt-5.4-mini: Acc = 0.671 vs. 0.542) and up to 0.090 points for assignment (e.g., gpt-5.5: Acc = 0.973 vs. 0.883), indicating that LOO in-context examples drawn from GitHub-native issues are more effective than augmenting the prompt with retrieved Bugzilla context. For invalid and duplicate detection, RAG produces mixed results: gpt-5.4 System B achieves zero false positives on invalid detection, and claude-haiku-4-5 System B improves duplicate detection performance, but these gains are model-specific and come at the cost of reduced recall or increased false positives in other models. Overall, the benefit of retrieval augmentation is limited and inconsistent across tasks and models.

7 Conclusion and Future Work

In this paper, we have presented TianoForge, an automated bug triage script for the TianoCore EDK II ecosystem that integrates four key triage tasks: invalid issue detection, duplicate detection, issue prioritization, and developer assignment. The proposed framework has combined LLMs, RAG, and domain-specific prompting to support end-to-end triage automation. Using a dataset of GitHub-native EDK II issues and historical Bugzilla reports, we have evaluated multiple state-of-the-art LLMs under both retrieval-free and retrieval-augmented settings. The experimental results have demonstrated that LLMs can effectively support several bug triage activities. The

findings have also shown that retrieval augmentation does not consistently improve performance and may, in some cases, introduce additional noise into the decision-making process. Furthermore, the proposed framework has substantially reduced triage latency compared to the current manual process, highlighting its potential to accelerate issue handling and improve developer productivity in large-scale Open Source Software (OSS) projects.

Beyond evaluating the proposed framework, we also used the LLM-assisted triage process to improve the quality of the GitHub-native dataset. All issues flagged as invalid or duplicate by the models were manually reviewed, resulting in the identification of three additional invalid issues and two additional duplicate issue pairs that were not originally labeled in the GitHub issue tracker. This finding highlights the practical value of the proposed framework not only as a triage automation tool but also as a mechanism for improving the quality and consistency of issue repositories.

For future work, we plan to expand the evaluation to additional OSS repositories to assess the generalizability of the proposed approach. We also intend to investigate more advanced retrieval strategies to improve contextual relevance. Finally, we plan to study confidence-aware bug triage by calibrating the reflective confidence scores generated by LLMs for each task in the integrated script, allowing the system to better distinguish between reliable predictions and cases requiring human intervention.

Software and Data Availability

Our research data and source code are publicly available [17, 20, 21].

Acknowledgments

This material is based upon work supported by the U.S. National Science Foundation (NSF) under Grant No. 2534021. Any opinions, findings, conclusions, or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of the NSF. Furthermore, in preparing this work, we used generative AI models and tools, including the OpenAI GPT and the Anthropic Claude models, to assist in generating and revising code and text.

References

- [1] Syed Nadeem Ahsan, Javed Ferzund, and Franz Wotawa. 2009. Automatic Classification of Software Change Request Using Multi-label Machine Learning Methods. In *2009 33rd Annual IEEE Software Engineering Workshop*. 79–86. doi:10.1109/SEW.2009.15
- [2] Iyad Alazzam, Ahmed Aleroud, Zainab Al Latifah, and George Karabatis. 2020. Automatic bug triage in software systems using graph neighborhood relations for feature augmentation. *IEEE Transactions on Computational Social Systems* 7, 5 (2020), 1288–1303.
- [3] John Anvik, Lyndon Hiew, and Gail C Murphy. 2006. Who should fix this bug?. In *Proceedings of the 28th international conference on Software engineering*. 361–370.
- [4] John Anvik and Gail C Murphy. 2011. Reducing the effort of bug report triage: Recommendations for development-oriented decisions. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 20, 3 (2011), 1–35.
- [5] Pamela Bhattacharya and Iulian Neamtii. 2010. Fine-grained incremental learning and multi-feature tossing graphs to improve bug triaging. In *2010 IEEE International Conference on Software Maintenance*. IEEE, 1–10.
- [6] Razvan Bocu, Alexandra Baicoianu, and Arpad Kerestely. 2023. An Extended Survey Concerning the Significance of Artificial Intelligence and Machine Learning Techniques for Bug Triage and Management. *IEEE Access* 11 (2023), 123924–123937. doi:10.1109/ACCESS.2023.3329732
- [7] Junjie Chen, Xiaoting He, Qingwei Lin, Hongyu Zhang, Dan Hao, Feng Gao, Zhangwei Xu, Yingnong Dang, and Dongmei Zhang. 2019. Continuous incident triage for large-scale online service systems. In *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 364–375.

Table 5: Best Performing Model per Task — Average over 3 Runs

Task	GT	Best Model	Sys	Acc	P+	R+	F1+	P-	R-	F1-	MP	MR	MF1	WP	WR	WF1
Bug Assignment	37 labeled	gpt-5.5	A	0.973	—	—	—	—	—	—	0.983	0.983	0.977	0.986	0.973	0.973
Prioritization	75 labeled	claude-sonnet-4-6	A	0.636	—	—	—	—	—	—	0.636	0.616	0.620	0.652	0.636	0.637
Dup. Detection	0 pairs dup.	claude-sonnet-4-6	A/B	0.973	1.000	1.000	1.000	1.000	0.973	0.987	1.000	0.987	0.493	1.000	0.973	0.987
Dup. Detection	2 pairs dup.	gpt-4o-mini / claude-sonnet-4-6 / claude-opus-4-7	A	0.960	1.000	0.500	0.667	0.973	1.000	0.986	0.986	0.750	0.826	0.973	0.986	0.977
Dup. Detection	2 pairs dup.	claude-sonnet-4-6	B	0.960	1.000	0.500	0.667	0.973	1.000	0.986	0.986	0.750	0.826	0.973	0.986	0.977
Inv. Detection	1 invalid	gpt-5.4	B	0.987	0.000	0.000	0.000	0.987	1.000	0.993	0.493	0.500	0.497	0.974	0.987	0.980
Inv. Detection	4 invalids	claude-sonnet-4-6	A	0.947	0.500	0.250	0.333	0.959	0.986	0.972	0.730	0.618	0.653	0.934	0.947	0.938

Table 6: Integrated Triage Framework Results — Average over 3 Runs (37 Labeled Issues, System A)

Task	Model	Acc	P+	R+	F1+	P-	R-	F1-	Mac-F1	Wgt-P	Wgt-R	Wgt-F1
Invalid Detection	claude-sonnet-4-6	0.973	0.000	1.000	0.000	1.000	0.973	0.986	0.493	1.000	0.973	0.986
Duplicate Detection	claude-sonnet-4-6	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000
Prioritization	claude-sonnet-4-6	0.559	—	—	—	—	—	—	0.487	0.565	0.559	0.547
Bug Assignment	gpt-5.5	0.955	—	—	—	—	—	—	0.946	0.959	0.955	0.949

Table 7: Runtime Summary — Average over 3 Runs (System A, 37 Labeled Issues)

Task	Time (s)	Time (min)
Invalid Detection	82.69	1.38
Duplicate Detection	88.32	1.47
Priority Classification	101.80	1.70
Bug Assignment	145.39	2.42
Inference (LLM only)	419.20	6.99
Setup (CSV + index)	5.40	0.09
Total	424.61	7.08

Models Activated: claude-sonnet-4-6 (Tasks 1–3), gpt-5.5 (Task 4).

- [8] Deepshikha Chhabra and Raman Chadha. 2025. ReBaRF-Bug: A Multi-Stage Augmentation and Deep Learning-Enhanced Approach for Automated Bug Classification and Prioritization. In *2025 2nd International Conference on Research Methodologies in Knowledge Management, Artificial Intelligence and Telecommunication Engineering (RMKMATE)*. IEEE, 1–8.
- [9] Anh-Hien Dao and Cheng-Zen Yang. 2023. Automated priority prediction for bug reports using comment intensiveness features and SMOTE data balancing. *International Journal of Software Engineering and Knowledge Engineering* 33, 03 (2023), 415–433.
- [10] Angela Fan, Beliz Gokkaya, Mark Harman, Mitya Lyubarskiy, Shubho Sengupta, Shin Yoo, and Jie M. Zhang. 2023. Large Language Models for Software Engineering: Survey and Open Problems. In *2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)*. 31–53. doi:10.1109/ICSE-FoSE59343.2023.00008
- [11] Murun Ganzorig, Jinfeng Ji, and Geunseok Yang. 2025. Security Bug Report Classification via Cross-Project Similarity-Based Data Augmentation and Deep Learning Models. *IEEE Access* (2025).
- [12] Huoliang He and ShunKun Yang. 2021. Automatic bug triage using hierarchical attention networks. In *2021 IEEE 21st International Conference on Software Quality, Reliability and Security Companion (QRS-C)*. IEEE, 1043–1049.
- [13] Kiana Kiashemshaki, Arsham Khosravani, Alireza Hosseinpour, and Arshia Akhavan. 2025. Automated Bug Triage using Instruction-Tuned Large Language Models. *arXiv preprint arXiv:2508.21156* (2025).
- [14] Atish Kumar Dipongkor. 2024. An ensemble method for bug triaging using large language models. In *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings*. 438–440.
- [15] Jaehyung Lee, Kisun Han, and Hwanjo Yu. 2022. A light bug triage framework for applying large pre-trained language model. In *Proceedings of the 37th IEEE/ACM international conference on automated software engineering*. 1–11.
- [16] Jinyang Liu, Shilin He, Zhuangbin Chen, Lique Li, Yu Kang, Xu Zhang, Pinjia He, Hongyu Zhang, Qingwei Lin, Zhangwei Xu, et al. 2023. Incident-aware duplicate ticket aggregation for cloud systems. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2299–2311.
- [17] Makubacki. 2026. bugzilla2github: TianoCore Bugzilla XML Archive. https://github.com/makubacki/bugzilla2github/tree/bz2gh_tianocore/final_xmils. Accessed: 2026-06-01.
- [18] Gloria Phillips-Wren and Anne Håkansson. 2025. Towards Using Prompt Engineering in Large Language Models to Assist Decision Making. *Procedia Computer Science* 270 (2025), 5225–5238.
- [19] N. Serrano and I. Ciordia. 2005. Bugzilla, ITracker, and other bug trackers. *IEEE Software* 22, 2 (2005), 11–13. doi:10.1109/MS.2005.32
- [20] Nazanin Siavash, Terrance Boulton, and Armin Moin. 2026. EDK II Bug Issue Dataset: A GitHub-Sourced Collection of Closed EDK II Bug Reports. doi:10.7910/DVN/DXDF7U
- [21] Nazanin Siavash, Terrance E. Boulton, and Armin Moin. 2026. TianoForge: Automated Bug Triage for TianoCore EDK II. <https://github.com/TianoShield/TianoForge>. GitHub repository.
- [22] Nazanin Siavash and Armin Moin. 2025. Model-Driven Quantum Code Generation Using Large Language Models and Retrieval-Augmented Generation. In *2025 ACM/IEEE 28th International Conference on Model Driven Engineering Languages and Systems (MODELS)*. 260–266. doi:10.1109/MODELS67397.2025.00031
- [23] Xiangchen Song, Yulin Huang, Jinxi Guo, Yuchen Liu, and Yaxuan Luan. 2025. Multi-scale feature fusion and graph neural network integration for text classification with large language models. *arXiv preprint arXiv:2511.05752* (2025).
- [24] Yanqi Su, Zhenchang Xing, Xin Peng, Xin Xia, Chong Wang, Xiwei Xu, and Liming Zhu. 2021. Reducing bug triaging confusion by learning from mistakes with a bug tossing knowledge graph. In *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 191–202.
- [25] TianoCore Contributors. 2025. EDK II Issue #11747. <https://github.com/tianocore/edk2/issues/11747>. GitHub issue, accessed June 5, 2026.
- [26] Qasim Umer, Hui Liu, and Inam Illahi. 2019. CNN-based automatic prioritization of bug reports. *IEEE Transactions on Reliability* 69, 4 (2019), 1341–1354.
- [27] Zexin Wang, Jianhui Li, Minghua Ma, Ze Li, Yu Kang, Chaoyun Zhang, Chetan Bansal, Murali Chintalapati, Saravan Rajmohan, Qingwei Lin, et al. 2024. Large language models can provide accurate and interpretable incident triage. In *2024 IEEE 35th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 523–534.
- [28] Xin Xia, David Lo, Ying Ding, Jafar M Al-Kofahi, Tien N Nguyen, and Xinyu Wang. 2016. Improving automated bug triaging with specialized topic model. *IEEE Transactions on Software Engineering* 43, 3 (2016), 272–297.
- [29] Jifeng Xuan, He Jiang, Zhilei Ren, Jun Yan, and Zhongxuan Luo. 2010. Automatic Bug Triage using Semi-Supervised Text Classification. In *SEKE*. 209–214.
- [30] Anurag Yadav and Santosh Singh Rathore. 2024. A hierarchical attention networks based model for bug report prioritization. In *Proceedings of the 17th Innovations in Software Engineering Conference*. 1–5.
- [31] Doohee You, Andy Parisi, Zach Vander Velden, and Lara Dantas Inojosa. 2025. LLM-as-classifier: Semi-Supervised, Iterative Framework for Hierarchical Text Classification using Large Language Models. *arXiv preprint arXiv:2508.16478* (2025).